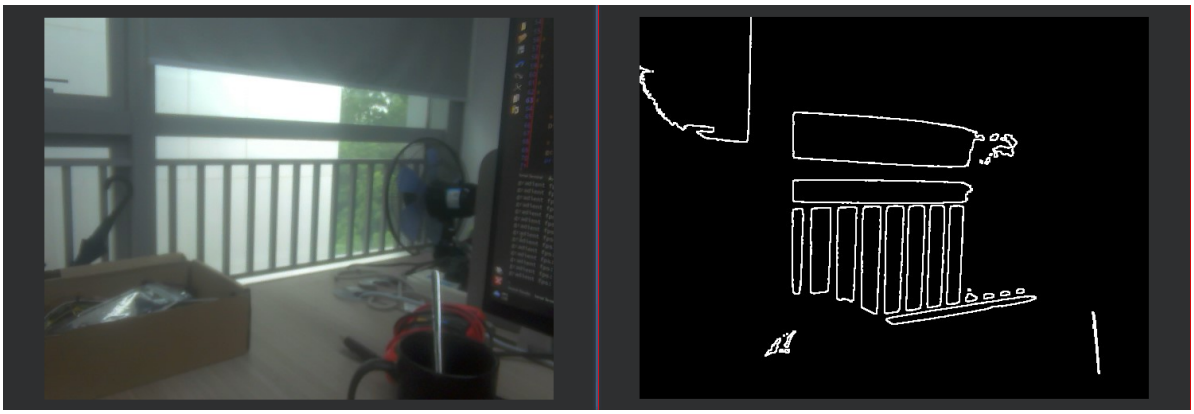
梯度变换

梯度变换
- 例程效果
- 原理讲解
  - 什么是图像梯度
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置梯度操作参数
    - 图像处理与梯度运算
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/18.rgb888_gradient.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像梯度处理。



原理讲解

什么是图像梯度

图像梯度（Morphological Gradient）是一种形态学图像处理操作，用于提取图像中对象的边界或边缘信息。它通过计算膨胀（Dilation）和腐蚀（Erosion）操作的结果之差来实现。梯度操作在图像处理中常用于边缘检测或轮廓提取。

- **操作原理**：形态学梯度定义为膨胀结果与腐蚀结果的差值，即 `Gradient = Dilate(Image) - Erode(Image)`。膨胀操作会扩展白色区域（前景）的边界，而腐蚀操作会收缩白色区域的边界，两者相减后，得到的是边界区域，即原始图像中白色区域的边缘部分。
- **效果**：梯度操作能够突出图像中对象的轮廓或边界，适用于二值图像或灰度图像。结果图像中，边界区域通常显示为白色（高值），而其他区域为黑色（低值）。这种操作可以看作是一种简单的边缘检测方法，尤其在二值图像中效果明显。
- **应用场景**：形态学梯度常用于边缘提取、对象轮廓检测以及作为其他复杂形态学操作的中间步骤。例如，在目标检测中用于分离对象的边界，在医学图像处理中用于提取结构边缘。

在梯度操作中，卷积核的大小和迭代次数会显著影响结果：较大的核和更多的迭代次数会导致更宽的边界区域。此外，二值化阈值（如果有）会决定哪些像素被视为前景（白色）或背景（黑色）。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *      # 摄像头接口 / Camera interface
from media.display import *     # 显示接口 / Display interface
from media.media import *       # 媒体资源管理器 / Media manager
import cv_lite                  # AI CV扩展模块 / AI CV extension module
import ulab.numpy as np
```

设置图像尺寸

```
image_shape = [480, 640]
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=30)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 30 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的形态学操作）。

初始化显示模块

```
Display.init(Display.VIRT, width=image_shape[1], height=image_shape[0],
to_ide=True)
```

初始化显示模块，使用虚拟显示驱动（`Display.VIRT`），分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置梯度操作参数

```
kernel_size = 3         # 卷积核尺寸（建议奇数）/ Kernel size (recommended odd)
iterations = 1          # 形态学迭代次数 / Morphology iterations
threshold_value = 150   # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold value (0=use
Otsu)
clock = time.clock()    # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：卷积核尺寸，用于梯度操作（膨胀和腐蚀），建议为奇数，值越大边界区域越宽。
- `iterations`：梯度操作的迭代次数，值越大边界效果越显著，但计算量增加。
- `threshold_value`：二值化阈值，用于在梯度操作前或后处理图像，值为 0 时使用 Otsu 自动阈值算法。
- `clock`：用于计算帧率。

图像处理与梯度运算

```
while True:
    clock.tick()

    # 获取图像并转为 ndarray / Capture image and convert to ndarray
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 调用梯度操作（Gradient = 膨胀 - 腐蚀）/ Call morphological gradient (dilate -
erode)
    result_np = cv_lite.rgb888_gradient(image_shape, img_np, kernel_size,
iterations, threshold_value)

    # 构造灰度图像用于显示 / Construct grayscale image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=result_np)

    # 显示图像 / Show image
    Display.show_image(img_out)

    # 垃圾回收并打印帧率 / Garbage collect and print FPS
    gc.collect()
    print("gradient fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **梯度运算处理**：调用 `cv_lite.rgb888_gradient()` 函数执行形态学梯度操作（内部实现为膨胀减去腐蚀），返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `kernel_size`：卷积核尺寸，值越大提取的边界区域越宽，适合处理较大的对象或需要更明显的边缘；建议使用奇数（如 3、5、7），从 3 开始测试以平衡细节和边界宽度。
- `iterations`：迭代次数，值越大梯度效果（边界宽度）越强，但可能导致过度强调边界并丢失细节；建议从 1 开始，根据图像特性逐步增加。
- `threshold_value`：二值化阈值，值越高需要更亮的像素才能被视为前景，值为 0 时使用 Otsu 自动阈值；建议根据图像亮度从 50 到 150 测试，或设置为 0 以自动适应。